

Smart Wall Paint Visualizer

A MEAN Stack Virtual Room Paint & Design Preview Application

Project Report

Full Stack Web Development Course Project

Collaborating Organization: Unified Mentor

Reference: Behr Paint Visualizer

Name: Pearl Parmar

1. INTRODUCTION

Choosing the right wall paint colour is a critical decision in home interiors. Customers typically rely on imagination, physical sample cards, or in-store displays, none of which accurately reflect how a colour will look in their own room once lighting, furniture, and existing decor are taken into account. The Smart Wall Paint Visualizer is a web-based application that closes this gap: a user uploads a photo of their room, selects the wall they want to repaint, and previews the result in any colour before buying a single tin of paint.

This document describes the problem the project solves, its objectives and scope, the functional and non-functional requirements, the system architecture and technology choices, the data model, and the current state of the implementation.

2. PROBLEM STATEMENT

Customers face difficulty selecting suitable wall paint colours and designs because:

- They cannot accurately visualize the final painted look ahead of time
- Shade cards don't reflect a room's real lighting conditions
- Wrong paint choices lead to dissatisfaction and costly repainting
- There is no easy, user-friendly virtual preview tool for everyday customers

3. OBJECTIVES

PRIMARY OBJECTIVES

- Enable users to preview wall paint colours on their own room photos
- Reduce incorrect paint selection and repainting expenses
- Improve customer confidence in interior design decisions
- Provide a realistic, interactive visualization experience

SECONDARY OBJECTIVES

- Support interior designers and painters during client consultations
- Assist paint retailers in customer decision-making
- Encourage digital adoption in home-improvement planning
- Demonstrate a practical, end-to-end MEAN stack implementation

4. SCOPE

IN SCOPE (PHASE 1)

- Web-based room image upload
- Manual wall area selection (polygon and freehand brush)

- Virtual paint colour application with adjustable opacity/finish
- Before-and-after preview
- Save and download painted images
- User and admin roles

OUT OF SCOPE (PHASE 1)

- Automatic AI-based wall detection
- AR/VR live camera preview
- Native mobile application
- Direct paint purchase and payment

5. FUNCTIONAL REQUIREMENTS

5.1 USER FEATURES

- User registration and login (JWT-based authentication)
- Upload room images (JPG/PNG)
- Select wall areas manually using a polygon or brush tool
- Apply paint colours with adjustable opacity for a realistic effect
- Preview before-and-after results
- Save and download final designs

5.2 DESIGN AND VISUALIZATION MANAGEMENT

- Colour palette management
- Support for solid colours and multiple wall regions per image
- Editable wall selections
- Multiple design trials per uploaded image

5.3 ADMIN FEATURES

- Manage paint colours and shade details
- Add, update, or remove design patterns
- View user activity and saved designs
- Maintain system configuration and user roles

6. NON-FUNCTIONAL REQUIREMENTS

- Simple and intuitive UI
- Responsive layout for desktop and tablet
- Fast image rendering and processing
- Secure handling of user data (hashed passwords, JWT auth)
- High system availability

7. TECHNOLOGY STACK (MEAN)

Layer	Technology	Purpose
Frontend	Angular 15+, HTML5 Canvas, SCSS, TypeScript	Single-page application, wall selection canvas tool
Backend	Node.js, Express.js, RESTful APIs	Application server and API layer
Auth	JWT (jsonwebtoken), bcryptjs	Stateless authentication, password hashing
Database	MongoDB (Mongoose ODM)	Users, projects, colours, patterns
File handling	Multer	Room photo upload and storage
Deployment	AWS / Vercel / Netlify / Render	Hosting frontend and backend

8. SYSTEM ARCHITECTURE

The application follows a standard three-tier MEAN architecture. The Angular client communicates with an Express REST API over HTTPS, authenticating requests with a JSON Web Token attached as a Bearer header. The API layer validates and processes requests, then reads from and writes to MongoDB via Mongoose models. Uploaded room photos are stored on disk (or a cloud object store in production) and served through the API.

Component	Responsibility
Angular client	Renders pages, runs the canvas-based wall selection/paint tool, calls the REST API
AuthInterceptor	Attaches the JWT to every outgoing request; redirects to login on 401
Express API	Auth, project, colour, pattern, and admin routes; input validation; file upload handling
Auth middleware	Verifies JWT, exposes req.user, enforces admin-only routes

Component	Responsibility
MongoDB	Persists Users, Projects (wall selections + preview), Colors, Patterns

9. DATA MODEL

USER

- name, email, passwordHash, role (user / admin)
- favoriteColors, favoritePatterns (references)

PROJECT

- owner (User reference), title
- originalPhotoUrl, finalPreviewUrl
- wallSelections: array of { type: polygon/brush, points, brushSize, color, opacity }
- status (draft / saved), timestamps

COLOR

- name, hex, rgb, brand, finish (matte / satin / glossy / eggshell), categoryTags

PATTERN

- name, description, category, imageUrl, scaling

10. API SUMMARY

Method	Route	Description
POST	/api/auth/register	Create a new account
POST	/api/auth/login	Log in and receive a JWT
POST	/api/projects/upload	Upload a room photo, create a project
GET	/api/projects	List the current user's saved rooms

Method	Route	Description
PUT	/api/projects/:id	Save wall selections and the final preview
GET	/api/colors	List available paint colours
GET	/api/admin/stats	KPI dashboard numbers (admin only)

11. USER FLOW

- User registers or logs in
- User uploads a room photo
- User traces a wall (polygon) or paints over it (brush)
- User picks a paint colour and adjusts opacity
- User toggles before/after to compare, then saves the design
- User downloads the final image or revisits it later from Saved Designs

12. APPLICATION WALKTHROUGH

The screens below were captured from the working prototype and the application's UI mockups, built to the same design system as the final Angular app.

12.1 ROOM PHOTO, BEFORE PAINTING

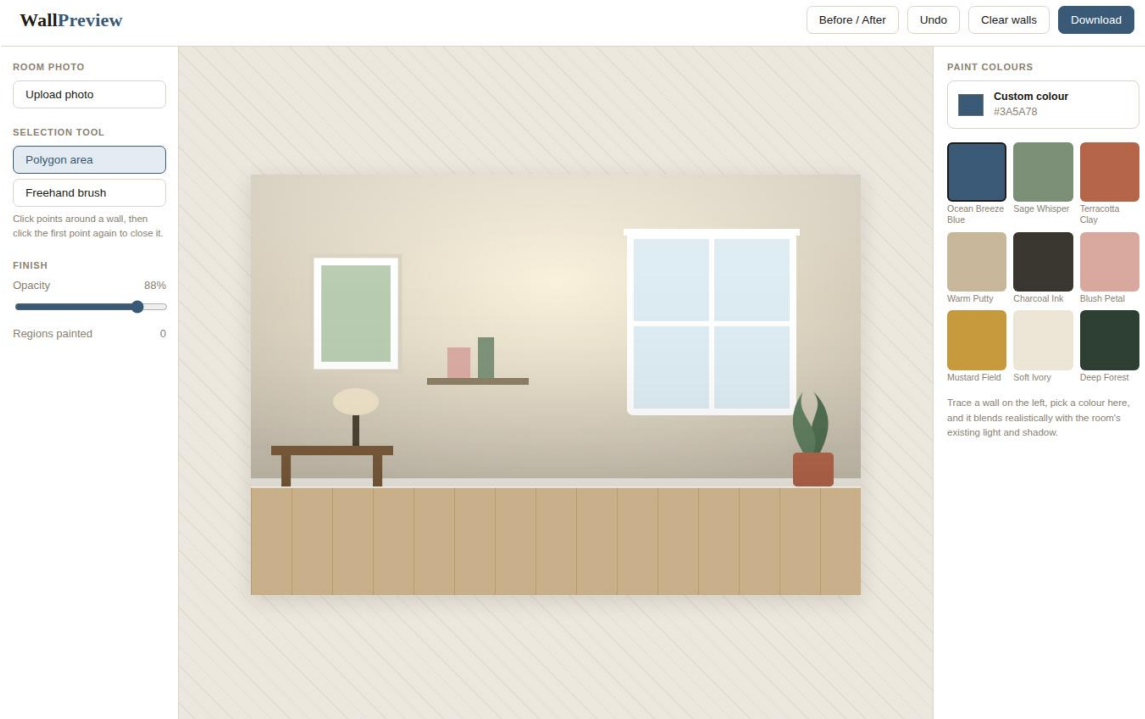


Figure 1 — A room photo uploaded, ready for wall selection.

12.2 WALL SELECTION AND PAINT APPLIED

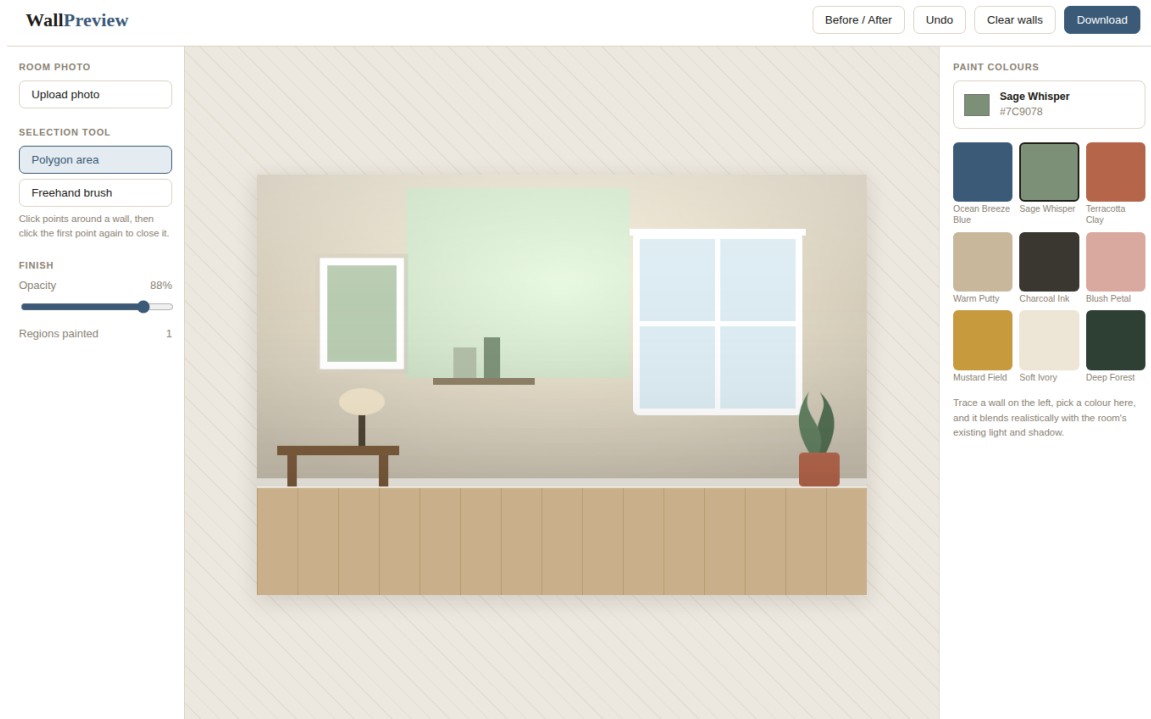


Figure 2 — A wall traced with the polygon tool and painted "Sage Whisper," blended realistically with the room's existing light and shadow.

12.3 LOGIN

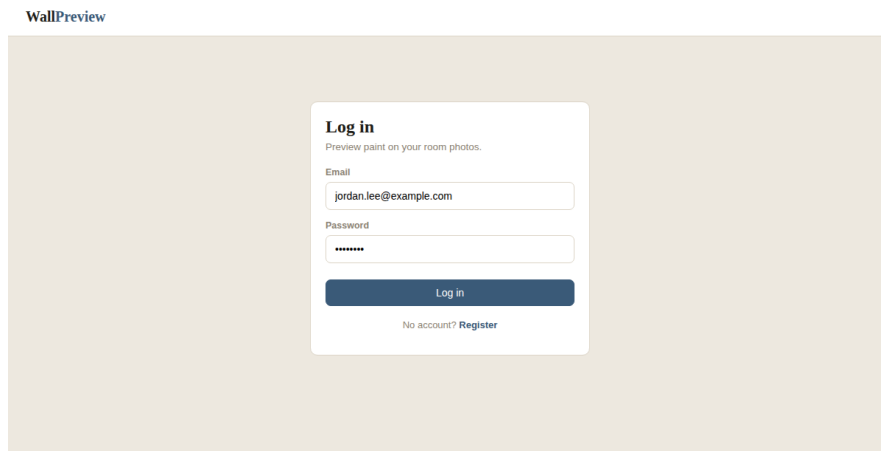


Figure 3 — Login page.

12.4 SAVED DESIGNS

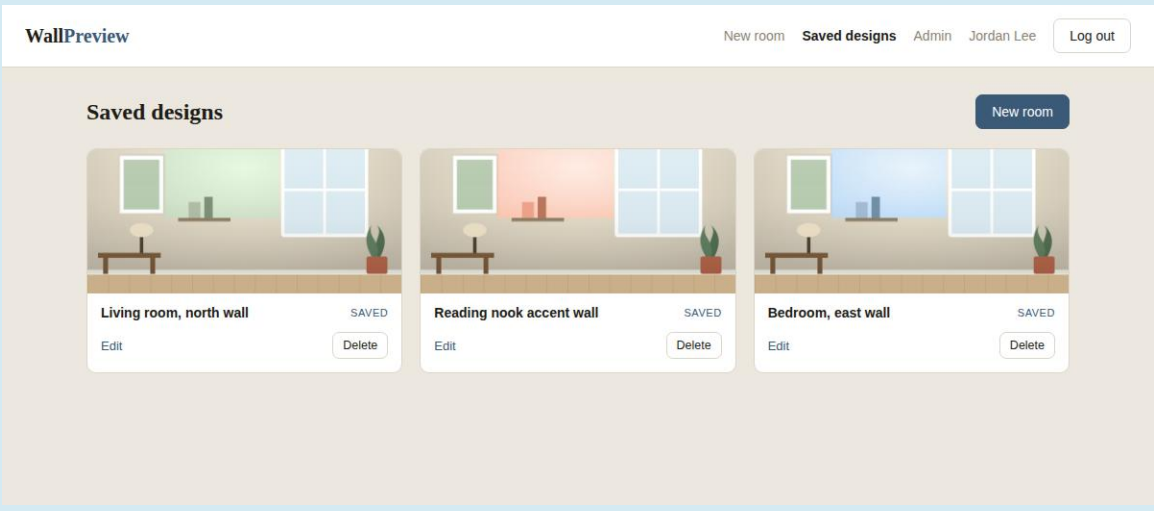


Figure 4 — A gallery of a user's saved room designs.

12.5 ADMIN PANEL

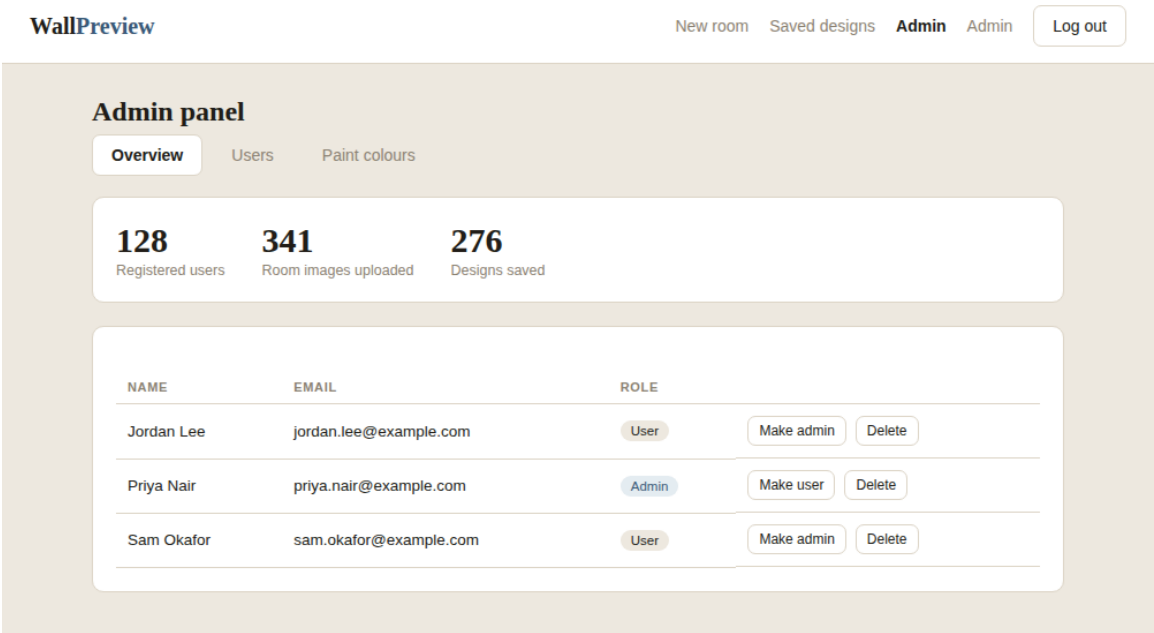


Figure 5 — KPI overview and user management in the admin panel.

13. ASSUMPTIONS AND CONSTRAINTS

ASSUMPTIONS

- Users upload clear, reasonably lit room photos
- Manual wall selection is acceptable for Phase 1
- Users have basic internet and browser access

CONSTRAINTS

- Image processing performance depends on photo size
- A limited set of colours and patterns is available initially
- No AI automation is included in this first version

14. DELIVERABLES

- This PRD/report documenting features, flows, and requirements
- Web application: Angular frontend + Node/Express backend, all in-scope features
- Admin dashboard for content and user management
- MongoDB schema definitions for users, projects, colours, and patterns
- Developer documentation: README, setup, and API reference
- Source code, organized and ready to run locally or deploy

15. FUTURE ENHANCEMENTS

- AI-based automatic wall detection
- Advanced textures and 3D room previews
- Lighting simulation (day/night modes)
- Integration with paint brands and retailers for direct purchase

16. CONCLUSION

The Smart Wall Paint Visualizer demonstrates a complete, end-to-end MEAN stack application that solves a genuine, everyday interior-design problem: knowing what a paint colour will actually look like on your own wall before you buy it. The implementation covers user authentication, manual wall selection with two input tools, realistic colour compositing, project persistence, and an admin panel for content and user management — fulfilling the in-scope functional and non-functional requirements defined in this report.